

Chapter 12B: Matching Game Continued

Welcome back to our Matching Game example. As you will recall in our last lesson we learned how to manage multiple forms and pass parameters between them. We also learned how to simultaneously have those multiple forms open or alternatively use the Hide, ShowDialog sequence to have only a single form open with forced focus.

In this, the second half of our Matching Game example, we will learn how to utilize random number generators and timers. Let's jump to the inside and look at some code.

Congratulations, your Matching Game project should now have two Forms talking with one another, a big step. Now it is time to build the playing board and add one more form.

We begin by changing the Size property of our MatchingGame Form to 600,600.

Next add a TableLayoutPanel to your Form and change its Size property to 550,475 and Location property to 20,80. Go ahead and Anchor that guy to all four sides of the form. Change the BackColor property of that guy to Pale Turquoise which you can find under the Web tab. Next please set the CellBorderStyle property to InsetDouble. That will add a 2 pixel width border around each of the table's cells.

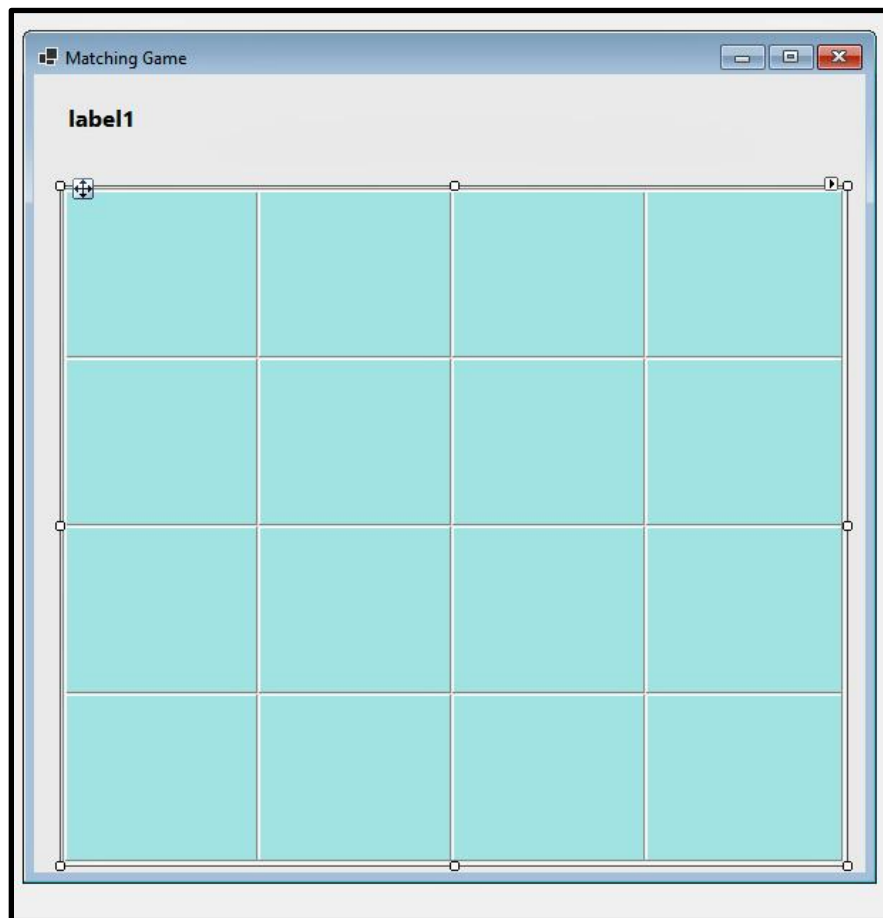
MatchingGame		
	Size	600,600

Next please change each of the ColumnCount and RowCount properties to 4. That should give you a very nice 4x4 grid for our playing board. Please go ahead and make each of those cells equal in size by opening the quick menu wizard and adjusting each of their relative sizes to 25%. By the way, the programmer who designed that Edit Rows and Columns form should be shot. Talk about unintuitive, please go ahead and set the percentages such that they don't add up to 100%, pure chaos ensues. Who is doing QA at Microsoft these days? Anyway, when properly completed, the current form should look like this.

(Figure 12B.1)

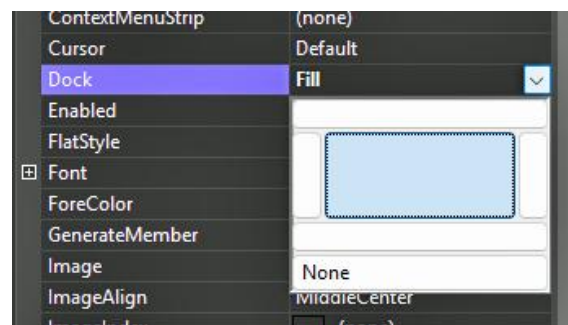
TableLayoutPanel1		
	Size	550,475
	Location	20,80
	Anchor	All 4 Sides
	BackColor	Pale Turquoise
	CellBorderStyle	InsetDouble
	ColumnCount	4 equal 25%
	RowCount	4 equal 25%

Figure 12B.1: MatchingGame Form with TableLayoutPanel



Next make certain the TableLayoutPanel is selected and then add a Label to your Form. Change the Label's AutoSize property from its default value of true to false. The AutoSize property determines whether the Label will automatically resize as a function of the size of the text or image displayed on the Label. Normally this property would be set to true, but we are going to toggle it off because we know the size of the images to be displayed on the Label.

Next set the Dock property of your Label to Fill. Do this by selecting the center box from the DropDown image. This Label is ultimately going to display one of the game piece images. Setting the Dock property to Fill ensures that if the playing board is resized by the user the playing pieces resize accordingly.



Next please change the TextAlign property to MiddleCenter, again do this by selecting the center box from the DropDown image. Finally, please set the Font property to Webdings, bold and 48 point. Complete this first Label's properties by setting its Text property to a lower case c. Cool, apparently the Webding image associated with a lower case c is a square.

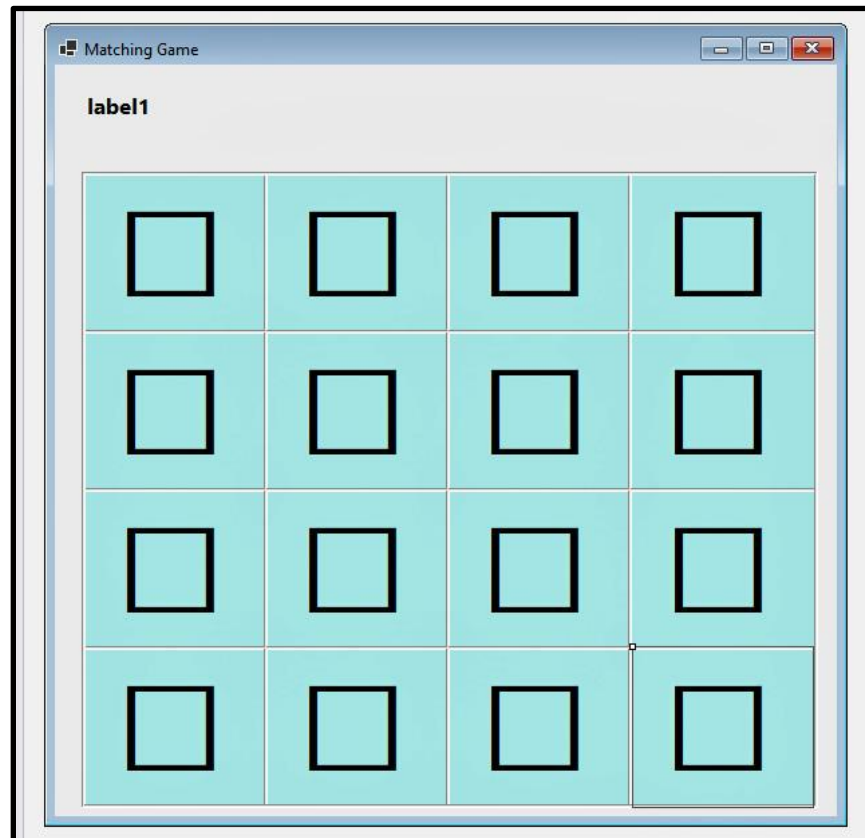
Label1		
	AutoSize	False
	Dock	Fill
	TextAlign	MiddleCenter
	Font	Webdings, Bold, 48 point
	Text	c

Here is the deal with the Webdings font. Instead of text, the Webdings font is actually a collection of simple images each associated with a letter in the English language. We are going to randomly assign each of those images to our table cells for our playing pieces.

Please now finish creating the playing board by copying and pasting that first Label into each subsequent cell of the TableLayoutPanel. Remember, the hot key combination for copy is Ctrl C and paste is Ctrl V. Just as an FYI undo is Ctrl Z.

Your current form should look like this. (Figure 12B.2)

Figure 12B.2: MatchingGame with Webdings Font



We now need to write the code which assigns one of the Webding images to each cell of our playing board ensuring that the assignments are done in random pairs. My version of the code is shown here. I expect some explanation is required as this is probably the most intense code we have written so far. (Figure 12B.3)

We start by creating an instance of the Random class named `r`. We next create an instance of the List class of type string and assign it pairs of lowercase letters starting with the letter `a` and ending with the letter `h`. Why these letters? Because there are 16 cells in our `TableLayoutPanel` each with a `Label` object in them. We are going to randomly put the Webding image associated with each of these letters into those 16 cells.

Figure 12B.3: Assign Webbing Images

```
4 references
public partial class MatchingGame : Form
{
    int user_prev_high_score;

    // create an instance of the Random class
    Random r = new Random();

    // create a list of string pairs from lower case a through h
    List<string> myIcons = new List<string>()
    { "a", "a", "b", "b", "c", "c", "d", "d", "e", "e", "f", "f", "g", "g", "h", "h" };
1 reference
    private void AssignIcons()
    {
        foreach (Control c in tableLayoutPanel1.Controls)
        {
            // instantiate each Control as Label class
            Label l = c as Label;

            // create an integer variable ran and assign it to the next available
            // random number smaller than the number of items in our string list
            // note, the myIcons.Count value will decrement with each loop cycle
            int ran = r.Next(myIcons.Count);

            // assign ran Webbing to Text property of Label l
            l.Text = myIcons[ran];

            // remove the current value from myIcons
            myIcons.RemoveAt(ran);
        }
    }

2 references
    private void MatchingGame_Load(object sender, EventArgs e)
    {
        AssignIcons();
    }
}
```

The AssignIcons routine is pure magic. The first thing to know is that the TableLayoutPanel has a property called Controls which contains a List of all the controls managed by the TableLayoutPanel. As such, the foreach(Control c in tableLayoutPanel.Controls) syntax is going to walk through each item in that List, which in our case is all those Labels.

Inside the loop we begin by casting each control to the Label type. The next line of code is where the real magic happens. int ran = r.Next(myIcons.Count) says create an integer type variable named ran and assign it the next available random number r where the maximum of that random number is the current count of items in the myIcons List. The next line is straight forward, assign the Text property of the current Label the ran index of the myIcons List. We conclude the loop by removing the currently assigned item from the myIcons List. Come on, that is some pretty cool code.

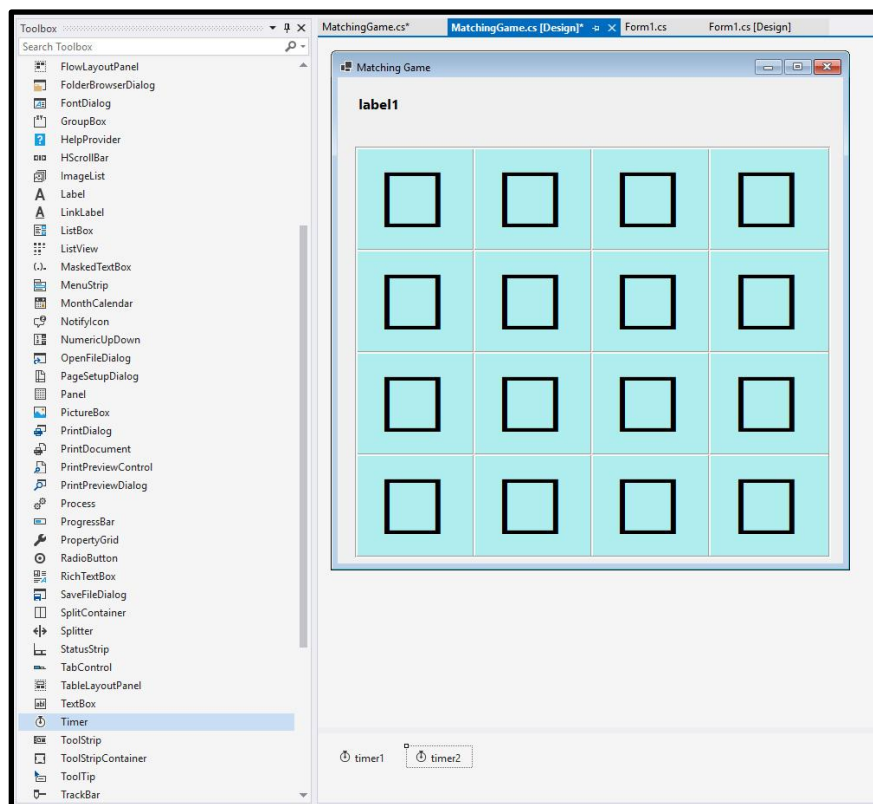
I next update the form's load routine by adding a call to the AssignIcons routine and we are in business.

The matching game is timed. The user has a maximum of 90 seconds to match all 8 pairs of images, or the game is halted and an appropriate message is delivered. Additionally, when a user selects a pair of images which do not match, they remain visible for 2 seconds thus

allowing the user to memorize the board. During those 2 seconds all clicks are ignored. The user with the lowest overall time is considered the winner.

To begin building those Timers please add two Timers to your Form. Check it out, the Timers appear below the actual Form in a little window. (Figure 12B.4) That is because Timers are never visible to the user. They work behind the scenes. We are going to use Timer1 to count down the 90 seconds allowed to finish the game and Timer2 will measure the 2 second interval non-matching pairs remain visible. Note, you may need to close a couple of panels at the bottom of your screen to see your Timers.

Figure 12B.4: Timers



Before we start coding our Timers please add a ProgressBar and another Label to your Form. The properties of those objects are as follows:

ProgressBar1		
	Location	290,20
	Maximum	90
	Minimum	0
	Size	200,25

Label17		
	Location	500,20
	Font	Segoe UI, Bold, 12
	Text	0
	Name	TimerLabel

Now onto our Timers. Select timer1 and note the very small number of properties and methods that guy has. That's because you generally only use one Timer property and one Timer method. The Timer property used most often is called the Interval. A Timer's Interval is measured in milliseconds and corresponds to the duration a Timer runs before raising a Tick event, the Timer's most common method. Please set timer1's Interval to 1000 milliseconds, i.e. timer1 will raise a Tick event every second.

This image (Figure 12B.5) shows the modifications we need to make to the Form's load event handler to support timer1. In particular, we initially set our ProgressBar's Value property to zero as well as the Text property of our TimerLabel to "0". As the name implies, a Timer's Start method initiates a Timer interval.

The code in the middle of that event handler is also pretty cute. To make the image of a Label invisible it is common practice to simply set its ForeColor property equal to its BackColor property. That of course is what is being done to each of our board pieces in the foreach loop. When the user clicks one of the cells in our grid we will revert its ForeColor back to Black.

By the way, the first and second variables shown in the image are of type Label. We will be using them shortly.

Figure 12B.5: Updated MatchingGame_Load

```
2 references
private void MatchingGame_Load(object sender, EventArgs e)
{
    AssignIcons();
    progressBar1.Value = 0;
    TimerLabel.Text = "0";
    first = null;
    second = null;

    foreach (Control c in tableLayoutPanel1.Controls)
    {
        Label l = c as Label;
        l.ForeColor = l.BackColor;
    }

    // start the 90 second game timer
    timer1.Start();
}
1 reference
```

I next wrote the `label_click` routine shown here. (Figure 12B.6) The code starts out by casting the sender parameter to a `Label` type. We know this is legal because that is all that is clickable on our Form. The routine then checks whether `timer2` is enabled or if the `ForeColor` of the current `Label` is set to black. If either of these conditions resolves to true the user has clicked a visible image, just ignore the click. The next if statement resolves to true for the user's first image selection. Under this condition we set the first variable equal to the current `Label` and set the current `Label`'s `ForeColor` to Black. The remaining code only executes when the user selects a second image. We begin by setting our second variable equal to the currently selected `Label` and then test whether the two selected images are equivalent. If they are equivalent, we leave them visible, if they are not visible we start `timer2` which will keep the selected images visible for 2 seconds. I know the comment reads check for a win which really isn't happening here yet, but it will in just a moment.

Also, please note how I assign the `label_click` event handler to each of our `Labels` in the form load routine with the `l.Click += label_click` syntax.

Figure 12B.6: label_click

```
foreach (Control c in tableLayoutPanel1.Controls)
{
    Label l = c as Label;
    l.ForeColor = l.BackColor;
    l.Click += label_click; // assign label_click to each label
}

// start the 90 second game timer
timer1.Start();
}

1 reference
private void label_click(object sender, EventArgs e)
{
    Label l = sender as Label;

    if (timer2.Enabled == true || l.ForeColor == Color.Black)
        // just ignore the user clicked while it was visible
        return;

    if (first == null)
    {
        first = l;
        l.ForeColor = Color.Black;
        return;
    }

    // if the flow made it here we are on second pick
    second = l;
    l.ForeColor = Color.Black;

    // check for win
    if (first.Text == second.Text)
    {
        // leave the images visible
        first.ForeColor = Color.Black;
        second.ForeColor = Color.Black;
        first = null;
        second = null;
    }
    else
    {
        // start timer 2
        timer2.Start();
    }
}
```

The timer2_Tick routine should be very easy for you by now. Stop the Timer, make the first and second images invisible and reset our Label variables to null. (Figure 12B.7)

Figure 12B.7: timer2_Click

```
0 references
private void timer2_Tick(object sender, EventArgs e)
{
    timer2.Stop();
    first.ForeColor= first.BackColor;
    second.ForeColor= second.BackColor;
    first = null;
    second = null;
}
```

The timer1_Tick event handler is even easier. Increment the progressBar and TimerLabels and if the progressBar reaches a value of 90 stop everything and give the user a message. (Figure 12B.8)

Figure 12B.8: timer1_Tick

```
0 references
private void timer1_Tick(object sender, EventArgs e)
{
    progressBar1.Value += 1;
    TimerLabel.Text = (1+progressBar1.Value).ToString();

    if (progressBar1.Value == 90)
    {
        timer1.Stop();
        timer2.Stop();
        MessageBox.Show("Sorry, time has expired");
    }
}
```

By the way, don't forget to set the interval on timer2 and attach both timer click events in the form load routine. (Figure 12B.9)

Figure 12B.9: Updated Form Load

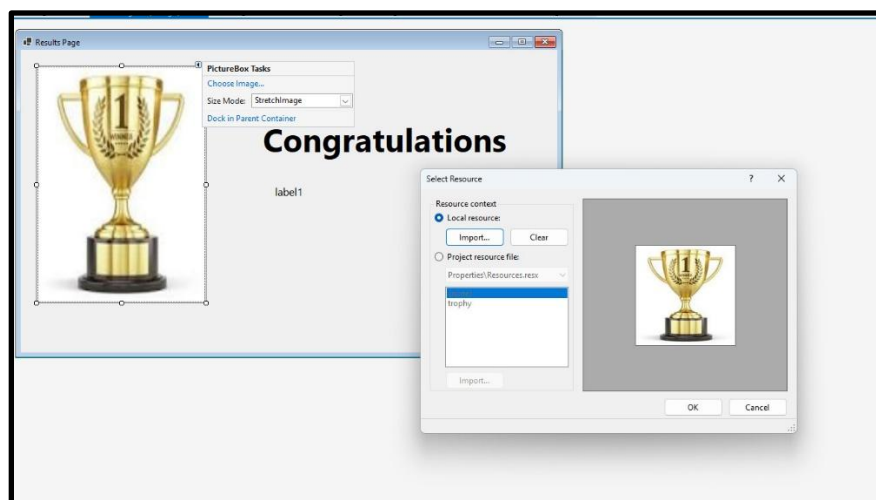
```
2 references
private void MatchingGame_Load(object sender, EventArgs e)
{
    AssignIcons();
    progressBar1.Value = 0;
    TimerLabel.Text = "0";
    first = null;
    second = null;
    timer1.Tick += Timer1_Tick;
    timer2.Interval = 1000;
    timer2.Tick += timer2_Tick;

    foreach (Control c in tableLayoutPanel1.Controls)
    {
        Label l = c as Label;
        l.ForeColor = l.BackColor;
        l.Click += label_click; // assign label_click to each label
    }

    // start the 90 second game timer
    timer1.Start();
}
```

Okay, time to add our third and final Form to this application. Please name that Form ResultsPage. Also please change the Text property of that Form to Results Page with a space between the two words. Finally, change the constructor for that page to accept an integer data type parameter named myScore and a string type parameter named myUsername. (Figure 12B.10)

Figure 12B.10: ResultsPage



On the design panel for our ResultsPage please add a PictureBox and change the Size property to 250,350 and the Location property to 25,25. From the quick wizard up in the upper righthand corner import an image as shown here. I chose this cool trophy. While you are at it, set the SizeMode property of the PictureBox to StretchImage.

PictureBox1		
	Location	25,25
	Size	250,350
	SizeMode	StretchImage
	Image	Your choice

Next please add two Labels to your form according to these specifications. Your final product should look something like this. (Figure 12B.11)

Label1		
	Name	Congrats
	Location	350,250
	Font	Segoe UI, Bold, 36
	Text	Congratulations!

Label2		
	Name	UserName
	Location	375,200
	Font	Segoe UI, Reg, 12

Figure 12B.11: Results Page Continued



I next added the CheckForWin routine shown here. A call to this routine was also added to the end of the label_click routine. (Figure 12B.12) Pretty straight forward stuff here just remember to parameterize that ResultsPage instantiation syntax.

Figure 12B.12: CheckForWin

```
// check for win
if (first.Text == second.Text)
{
    // leave the images visible
    first.ForeColor = Color.Black;
    second.ForeColor = Color.Black;
    first = null;
    second = null;
    CheckForWin();
}
else
{
    // start timer 2
    timer2.Start();
}

1 reference
private void CheckForWin()
{
    foreach (Control c in tableLayoutPanel1.Controls)
        if (c.ForeColor == Color.Black) return;

    // the user has won
    timer1.Stop();
    timer2.Stop();

    // open ResultsPage
    this.Hide();
    ResultsPage rp = new ResultsPage(progressBar1.Value, UserName.Text);
    rp.ShowDialog();
}
1 reference
```

Just one more small routine and this app is ready to rock and roll. We need to update the ResultsPage_Load routine to populate that UserName Label. No surprises here. (Figure 12B.13)

Figure 12B.13: ResultsPage_Load

```
namespace Chapter_12_Matching_Game
{
    4 references
    public partial class ResultsPage : Form
    {
        int myScore;
        string myUsername;

        1 reference
        public ResultsPage(int myScore, string myUsername)
        {
            InitializeComponent();
            this.myScore = myScore;
            this.myUsername = myUsername;
        }

        1 reference
        private void ResultsPage_Load(object sender, EventArgs e)
        {
            UserName.Text = "Congratulations " + myUsername + " " + myScore + " is a champion's score.";
        }
    }
}
```

That just about wraps it up. We of course could have made that ResultsPage much nicer but I'm sure you get the idea. Let's jump to the outside and review what we learned.

So, we had a lesson covering multiple Forms and moving data between them, random number generators and Timers. Some of the code was super easy while some was pretty tricky. That foreach loop where we were randomly assigning images to the gaming board labels was some pretty cool stuff. By the way, I sure hope you are writing all this code yourselves as we go along. That is the way you really learn.